

架构说明

设计目标

项目面向真实企业 Demo，要求当前可独立运行，同时后续能平滑接入 FastAPI/Flask 后端。核心设计原则是页面不直接依赖 Mock 文件，而是通过 `services -> api -> mock/real` 的分层获取数据。

本次架构 review 后增加了两类注册层：

- `features/algorithm`：算法展示域注册表，减少 Tab、Table 和统计代码重复。
- `features/charts`：图表注册表，页面只负责渲染，不硬编码每张图。

针对后端结构暂不稳定的问题，接口层还增加了 DTO adapter：

- `api/adapters/common.ts`：未知 payload 读取、类型转换、包装解包。
- `api/adapters/algorithmAdapter.ts`：将后端任意常见算法结构归一化为 `AlgorithmDataset`。

页面结构

```
App
  RouterProvider
    DemoLayout
      Header
      Sidebar
      Outlet
        HomePage
        DataImportPage
        ProjectSelectionPage
        AlgorithmPage
        ChartAnalysisPage
        ReportPreviewPage
```

所有页面路由均通过 `React.lazy` 懒加载，并由 `Suspense` 统一展示 `LoadingState`。菜单配置集中在 `src/router/navigation.tsx`，路由与布局不再各自维护一份导航数据。

`DemoLayout` 负责统一外壳：

- Header：Logo、项目名称、自动刷新时间、版本
- Sidebar：首页、数据导入、项目选择、算法展示、图表分析、报告预览
- Main：当前路由页面

数据流

```
Page
  -> hook/service
  -> api module
  -> adapter normalize
```

```
-> Mock data 或 Axios request  
-> typed response  
-> UI render
```

示例：

```
AlgorithmPage  
  -> useAlgorithmData  
  -> loadAlgorithmDataset  
  -> fetchAlgorithmResults  
  -> getMockAlgorithmDataset 或 GET /algorithm/results
```

分层职责

- **types/**：定义业务实体和接口数据结构。
- **mock/**：提供可演示的稳定样本数据。
- **api/**：负责 Axios、拦截器、接口路径、Mock/真实接口切换。
- **api/adapters/**：隔离后端 DTO，不让页面直接依赖后端字段命名。
- **services/**：给页面提供稳定业务函数，避免页面感知接口实现细节。
- **hooks/**：封装页面状态，例如加载、错误、当前时间。
- **pages/**：页面组装，不承载复杂算法。
- **features/**：业务域注册与组合逻辑，例如算法域配置、图表清单。
- **charts/**：图表容器和 option 构造函数。
- **utils/**：纯函数工具，例如统计、格式化、文件解析。
- **report/**：报告预览文档结构，后续可扩展 PDF 模板。

组件关系

```
PageTitle  
MetricCardGrid  
LoadingState  
ErrorState  
BaseChart  
AlgorithmResultPanel  
ChartSection  
ReportDocument
```

页面通过这些组件保持一致视觉和交互。图表统一走 **BaseChart**，内部处理 ECharts 初始化、setOption、ResizeObserver 和销毁。

扩展模型

算法展示扩展路径：

```
types -> mock -> statistics -> features/algorithm/domainRegistry -> AlgorithmPage  
自动渲染
```

图表扩展路径：

```
types/mock -> charts/options -> features/charts/chartRegistry -> ChartAnalysisPage  
自动渲染
```

这个结构降低了多人协作冲突：算法开发同学主要改类型、Mock、统计和 registry；图表开发同学主要改 option 和 chart registry；页面外壳基本不变。

为什么这样设计

- 后端未完成时，Mock 数据仍能覆盖真实工作流。
- 后续切真实 API 只需修改 `api/` 层或环境变量。
- 图表 option 与页面分离，便于算法字段调整。
- 算法域和图表使用 registry，新增 ECG、EEG、PPG 时不需要复制页面结构。
- 报告组件独立，便于未来扩展打印模板、PDF 服务或报告审核流程。
- TypeScript 类型集中定义，减少字段名漂移和接口联调成本。
- Adapter 层兼容后端字段未定阶段，后续联调时主要改数据映射，不改展示组件。
- Vite 构建使用页面懒加载、ECharts 按需注册和 vendor 分包，避免 Demo 首包无限膨胀。

Mock 到真实后端的切换

当前默认：

```
export const useMockApi = import.meta.env.VITE_USE MOCK !== 'false';
```

设置：

```
VITE_USE MOCK=false  
VITE_API_BASE_URL=http://localhost:8000/api
```

即可让 `api/` 层走真实请求。

API 层会统一处理：

- `ApiEnvelope<T>` 业务 code 非 0。
- FastAPI 422 `detail` 校验错误。
- Axios 网络错误。
- 每个请求自动附加 `X-Request-Id`。